Modern Database Systems

BITS Pilani

Prof. Ashish Narang
CSIS

innovate achieve lead

BITS Pilani

<SSZG507, Modern Database Systems>
Lecture No. 4

Contents

- **Distributed Transactions**
  - Transaction Processing
  - Two-Phase Commit (2PC)
- **Distributed Query Processing**
  - Query Execution Strategies
  - Distributed Query Optimization
- **CAP Theorem**
  - Consistency
  - Availability
  - Partition Tolerance
  - CAP Trade-offs
- **ACID vs BASE**
  - Comparing ACID and BASE
  - Choosing the Appropriate Approach

From Data Distribution to Distributed Operations

Consider a Simple Example

A bank maintains accounts at different database sites:

Site 1 – Delhi

- Account A
- Balance: ₹50,000

Site 2 – Mumbai

- Account B
- Balance: ₹30,000

Now consider:

Transfer ₹10,000 from Account A to Account B

This requires two operations:

$Debit(A, ₹, 10000)$ at Site 1

Credit(B,₹10,000) at Site 2

Questions that Arise

- What if one operation succeeds and the other fails?
- How do multiple sites commit or abort together?
- How are queries executed when required data resides at different sites?
- How do replicated sites maintain an appropriate level of consistency?
- What happens when sites cannot communicate because of a network partition?

Distributed Transactions

A **Distributed Transaction** is a transaction in which database operations are executed at **more than one site** in a distributed database system.

Local vs Distributed Transactions

- **Local Transaction**
  - Accesses data at **only one site**.
  - Executed and managed by the local DBMS.
- **Distributed (Global) Transaction**
  - Accesses or updates data at **multiple sites**.
  - Consists of multiple **subtransactions** executed at participating sites.
  - Requires coordination among the participating sites.
  - Must preserve the required **transaction properties** across the entire distributed operation.

Example

Consider a fund transfer:

T:Transfer ₹10,000 from Account A(Site:Delhi) to Account B (Site:Mumbai)

The transaction should behave as one logical unit even though its operations execute at multiple sites.

Anatomy of a Distributed Transaction

A distributed transaction involves **coordination between multiple participating sites**.

Key Components

- **Transaction Coordinator**
  - Coordinates execution of the **global transaction**.
  - Communicates with participating sites.
  - Coordinates the final **commit or abort decision**.
- **Participating Sites**
  - Sites containing data required by the transaction.
  - Execute their respective **sub transactions**.
- **Local Transaction Manager**
  - Manages transaction execution at an individual site.
  - Performs local concurrency control and recovery operations.
  - Communicates transaction status to the coordinator.
- **Local Database**
  - Stores the data accessed or modified by the local sub transaction.

Challenges in Distributed Transactions

- **Atomicity Across Sites**
  - All participating sites must agree on the final outcome.
  - A transaction should not be partially committed.
- **Site Failures**
  - A participating site may fail during transaction execution.
  - Other sites may continue to remain operational.
- **Communication Failures**
  - Messages between sites may be **delayed, lost, or interrupted**.
  - A site may be operational but temporarily unreachable.

- **Distributed Concurrency**
  - Transactions executing at different sites may access the same or related data.
  - Global consistency must be maintained.
- **Recovery**
  - After a failure, sites must determine the correct transaction state.
  - Recovery must account for actions already performed at other sites.
- **Coordination Overhead**
  - Additional messages and synchronization are required among participating sites.

Distributed Transaction Processing

A distributed transaction is executed across multiple database nodes but must be managed as **one logical transaction**.

Main Components

- **Transaction Coordinator** – manages and coordinates the overall transaction.
- **Participating Nodes** – execute local operations and report their status.

Processing Flow

Begin Transaction → Distribute Operations → Local Execution → Coordination → Global Commit / Abort

- Operations are executed at the respective participating nodes.
- Each node reports whether its operation was successful.
- The coordinator determines the final outcome.
- All participating nodes must follow the same final **commit or abort** decision.

Key Goal: Maintain a consistent transaction outcome across all participating nodes.

Two-Phase Commit (2PC) Protocol

Two-Phase Commit (2PC) is a coordination protocol used to ensure that all participating nodes in a distributed transaction reach a **common decision — COMMIT or ABORT**.

Two Phases of 2PC

Phase 1: Prepare / Voting Phase

- Coordinator sends a **PREPARE** request to all participating nodes.
- Each participant checks whether it can commit the transaction.
- Participants respond with **YES (Ready)** or **NO (Abort)**.

Phase 2: Commit / Decision Phase

- If **all participants vote YES** → Coordinator sends **COMMIT**.
- If **any participant votes NO** → Coordinator sends **ABORT**.
- All participants execute the final decision.

Two-Phase Commit (2PC) – Example

Example: Transfer ₹5,000 from Account A to Account B

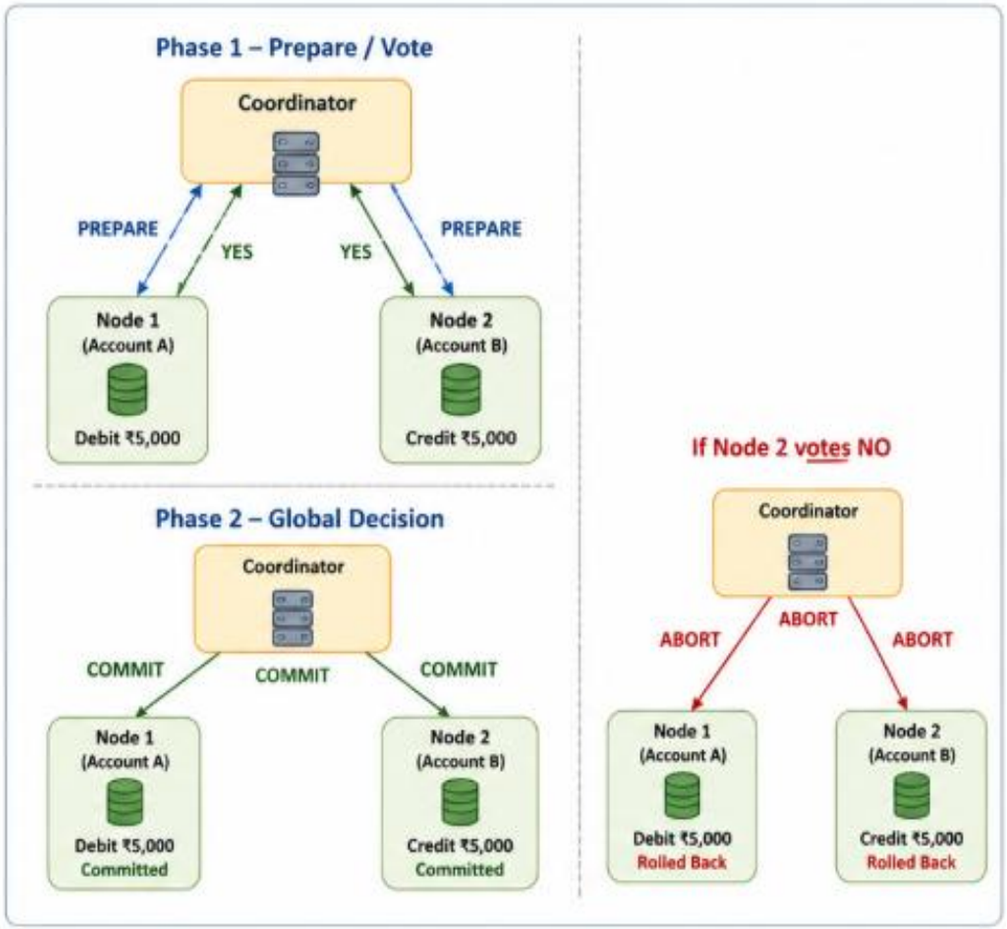
- **Node 1:** Stores Account A → Debit ₹5,000
- **Node 2:** Stores Account B → Credit ₹5,000
- **Coordinator:** Ensures both operations are committed together

Phase 1 – Prepare / Vote

Coordinator
|
├── PREPARE ──► Node 1 (Debit ₹5,000) ──► YES
|
└── PREPARE ──► Node 2 (Credit ₹5,000) ──► YES

Phase 2 – Global Decision

What if Node 2 votes NO?
Node 1: YES + Node 2: NO → Global ABORT → Both operations rolled back



Limitations of Two-Phase Commit (2PC)

1. Blocking Problem

- Participants may remain waiting if the coordinator fails after the **PREPARE** phase.
- Resources and locks may remain held until the decision is known.

2. Coordinator as a Critical Dependency

- Failure of the coordinator can delay transaction completion.
- Recovery may be required before participants can proceed.

3. Communication Overhead

- Multiple messages are exchanged between the coordinator and participants.
- Network latency increases transaction completion time.

4. Scalability and Performance

- Coordination becomes expensive as the number of participants increases.
- Less suitable for large-scale, highly distributed applications.

Distributed Query Processing

Distributed Query Processing involves executing a query when the required data is stored across **multiple sites or nodes** in a distributed database.

How is a Distributed Query Processed?

1. Query Decomposition

- Break the user query into smaller operations.
- Identify tables, conditions, joins, and required attributes.

2. Data Localization

- Determine **where the required data is stored**.
- Map query operations to the appropriate nodes/fragments.

3. Local Execution

- Relevant parts of the query are executed at individual nodes.

4. Result Integration

- Intermediate results are transferred and combined.
- Final result is returned to the user.

Example

```
SELECT C.Name, O.Amount
FROM Customer C JOIN Orders O
ON C.CustomerID = O.CustomerID;
```

If:
Customer → Node 1
Orders → Node 2
the DBMS must access both nodes and combine the results to execute the query.
Key Objective:
Minimize **data transfer, communication cost, and query execution time**.

Distributed Query Processing – Execution Strategies

A distributed query can often be executed in **multiple ways**. The DBMS selects an execution strategy that minimizes the overall cost.

Common Execution Strategies

1. Data Shipping

- Move required data to the node where the query is executed.
- Perform the operation after receiving the data.

2. Query Shipping

- Send the query/operation to the node where the data resides.
- Process data locally and return only the required result.

3. Hybrid Approach

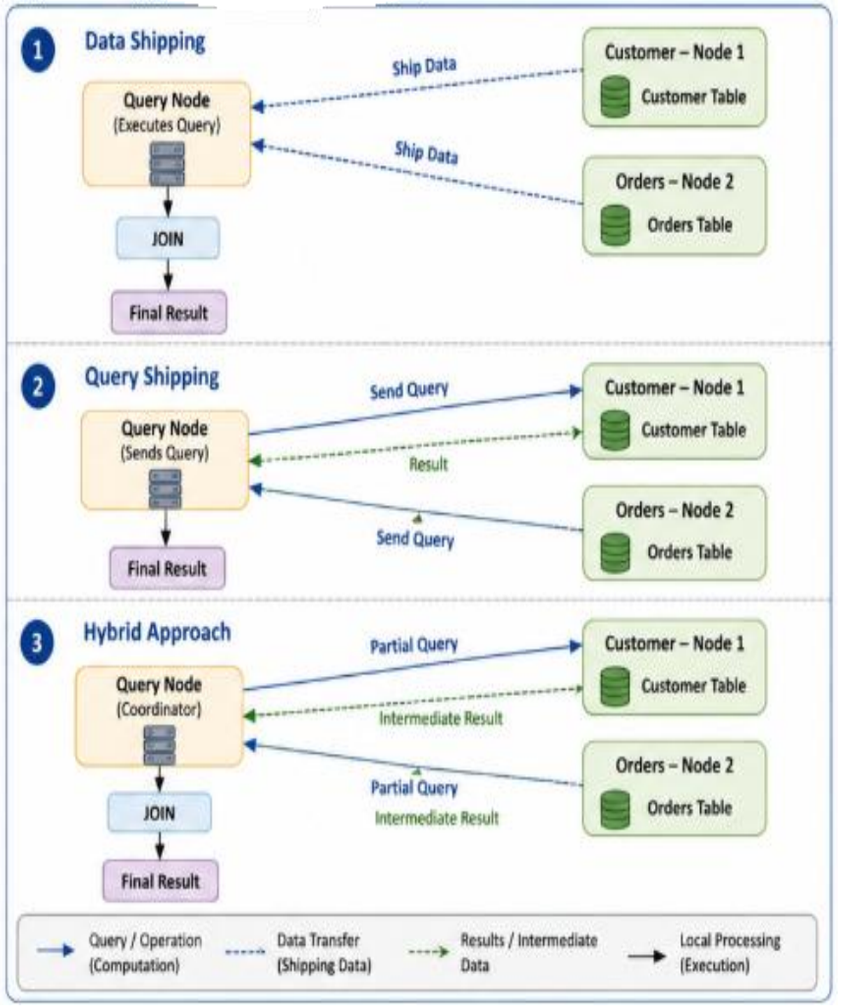
- Execute parts of the query at different nodes.
- Transfer only necessary intermediate results.

Major Cost Factors

Communication Cost | Data Volume | Local Processing Cost | Response Time

Key Idea:

Move computation closer to data whenever possible to reduce network communication.



Distributed Query Optimization

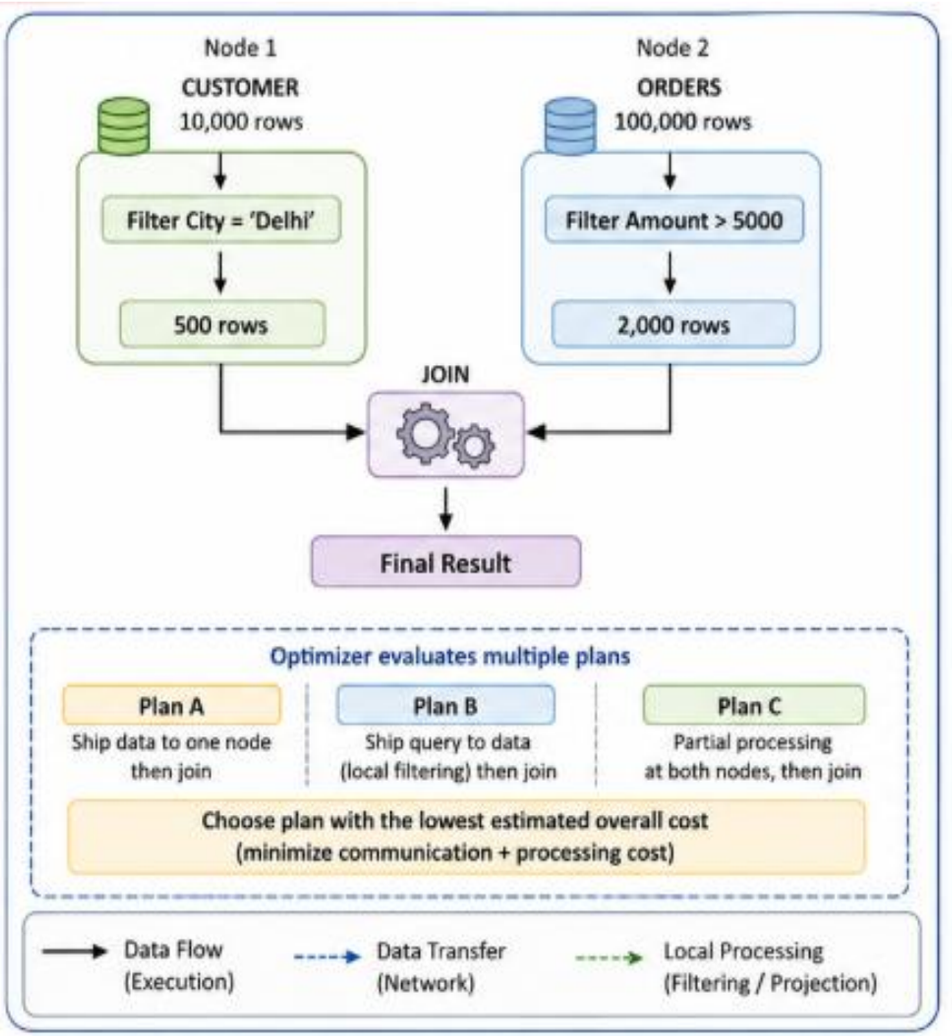
Distributed Query Optimization selects an efficient execution plan for a query involving data stored across multiple nodes.

Why is Optimization Required?

- A distributed query may have **multiple equivalent execution plans**.
- Different plans may involve significantly different amounts of **network communication and processing**.
- The optimizer selects a plan with the lowest estimated overall cost.

Key Optimization Decisions

- **Operation Location** – At which node should an operation execute?
- **Join Order** – In what sequence should distributed tables be joined?
- **Data Transfer** – Which data or intermediate results should be moved?
- **Local Processing** – Can filtering and projection be performed before data transfer?



CAP

Stands for Consistency (C) , Availability (A) and Partition Tolerance (P)

Triple constraint related to the distributed database systems

Consistency

- ✓ A read of a data item from any node results in same data across multiple nodes

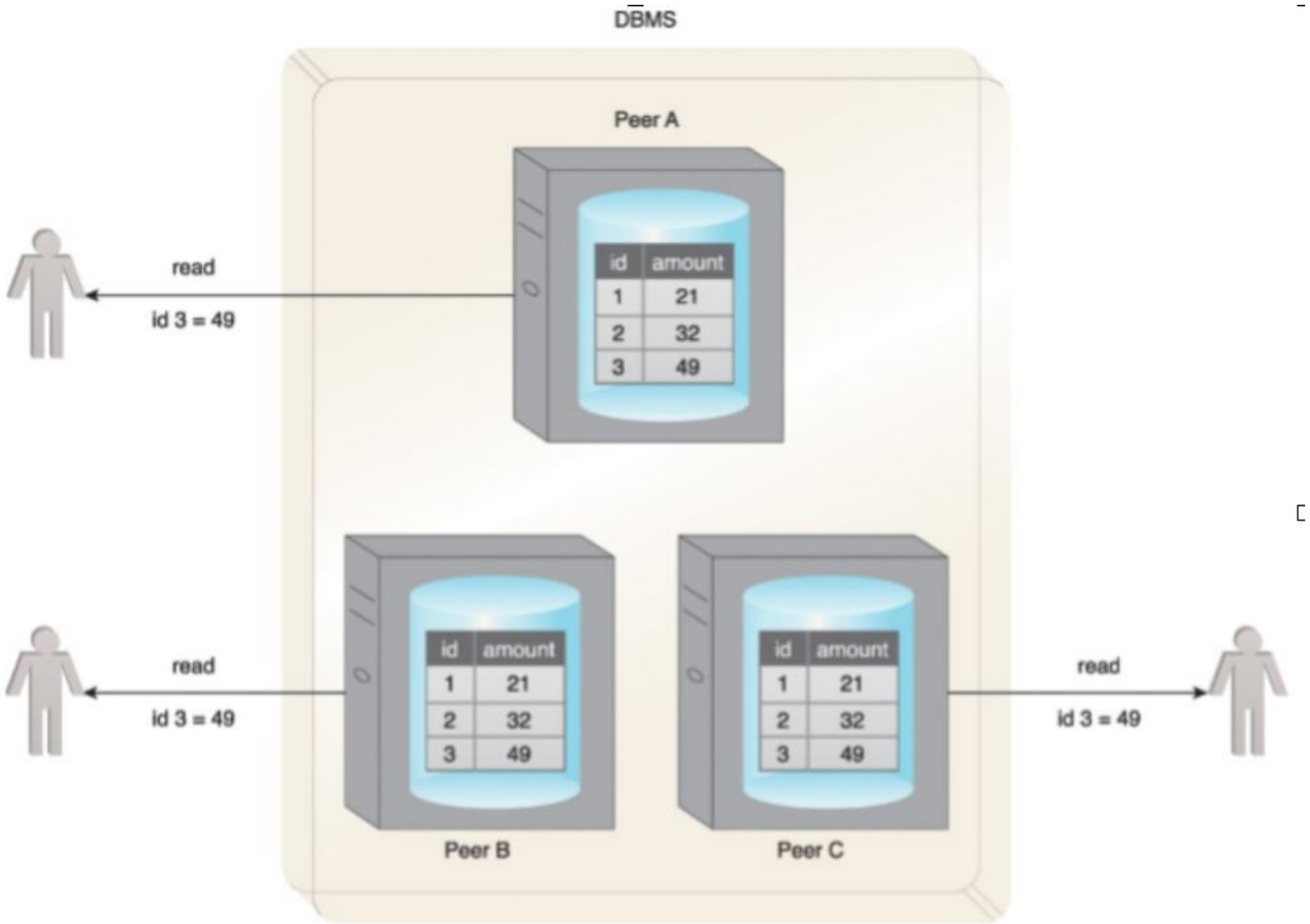
Availability

- ✓ A read/write request will always be acknowledged in form of success or failure in reasonable time

Partition tolerance

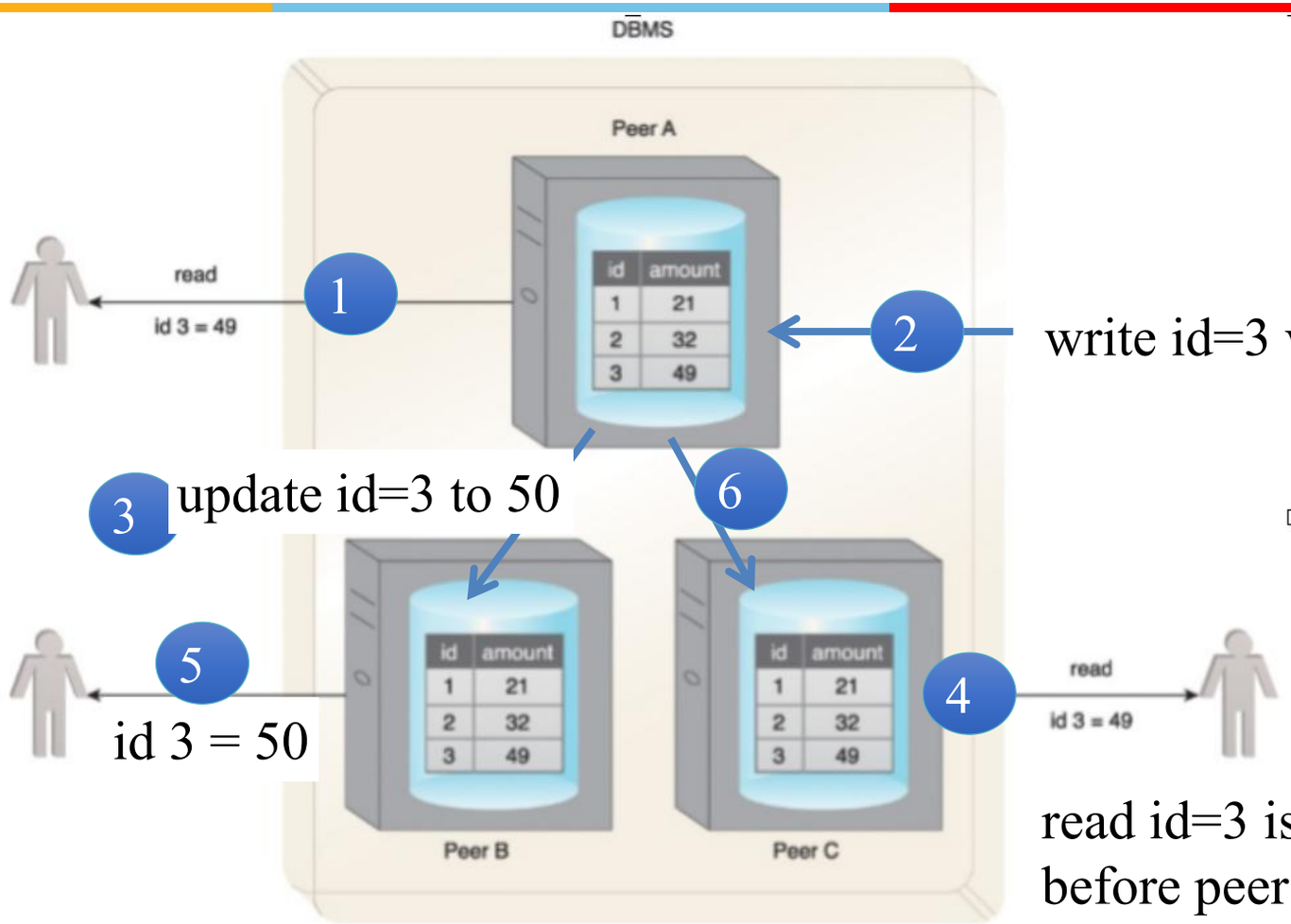
- ✓ System can continue to function when communication outages split the cluster into multiple silos and can still service read/write requests

Consistency



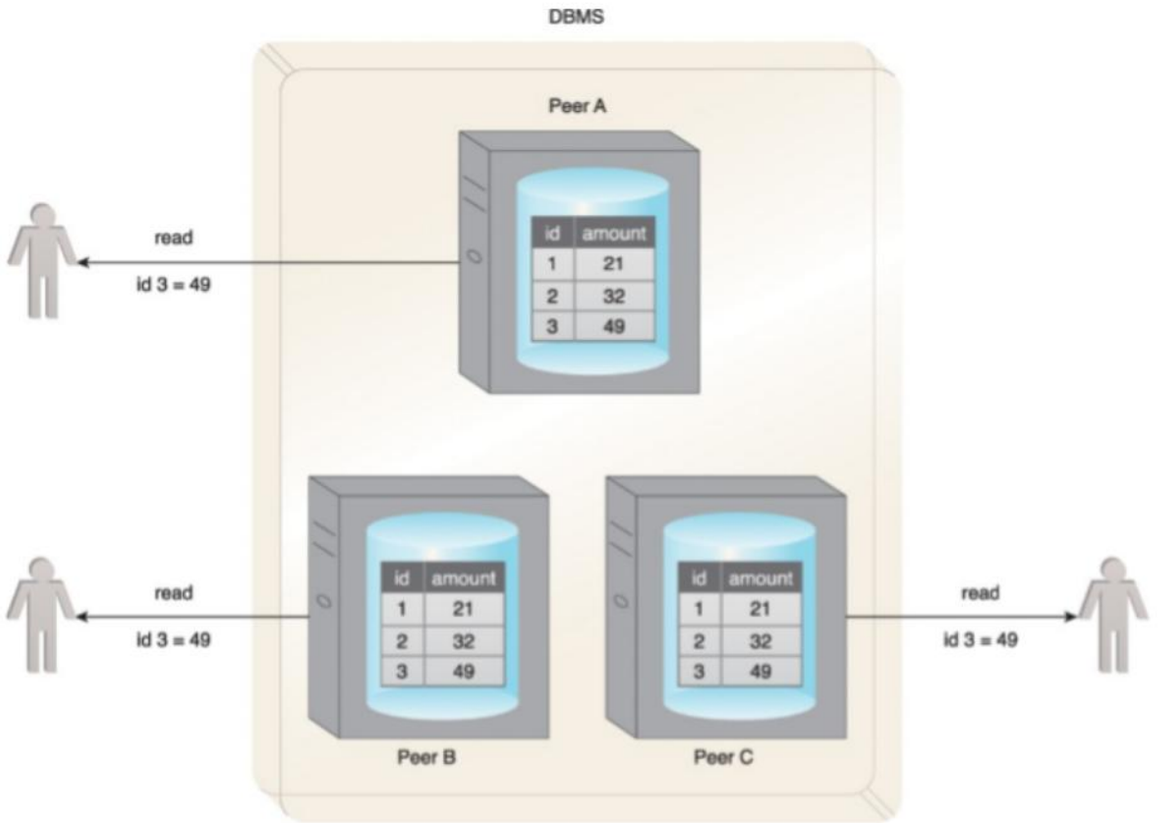
All users get the same value for the amount column even though different replicas are serving the record

When can it be in-consistent



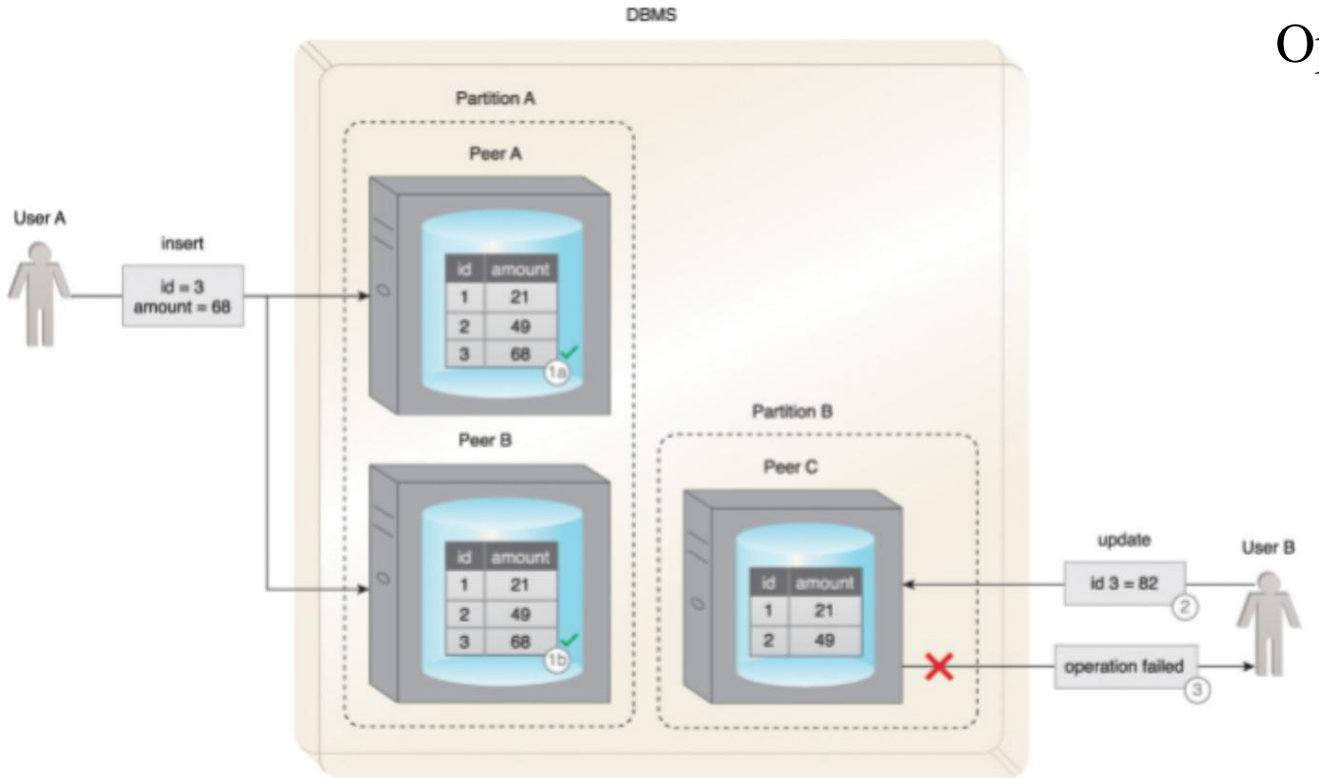
read id=3 is allowed
before peer C is updated
even though write "happens
before" read

Availability



- A request from a user to a peer always responds with a success or failure within a reasonable time.
- Say Peer C is disconnected from the network, then it has 2 options
  - Available: Respond with a failure when any request is made, or
  - Unavailable: Wait for the problem to be fixed before responding, which could be unreasonably long and user request may time out

P: Partition tolerance (1)

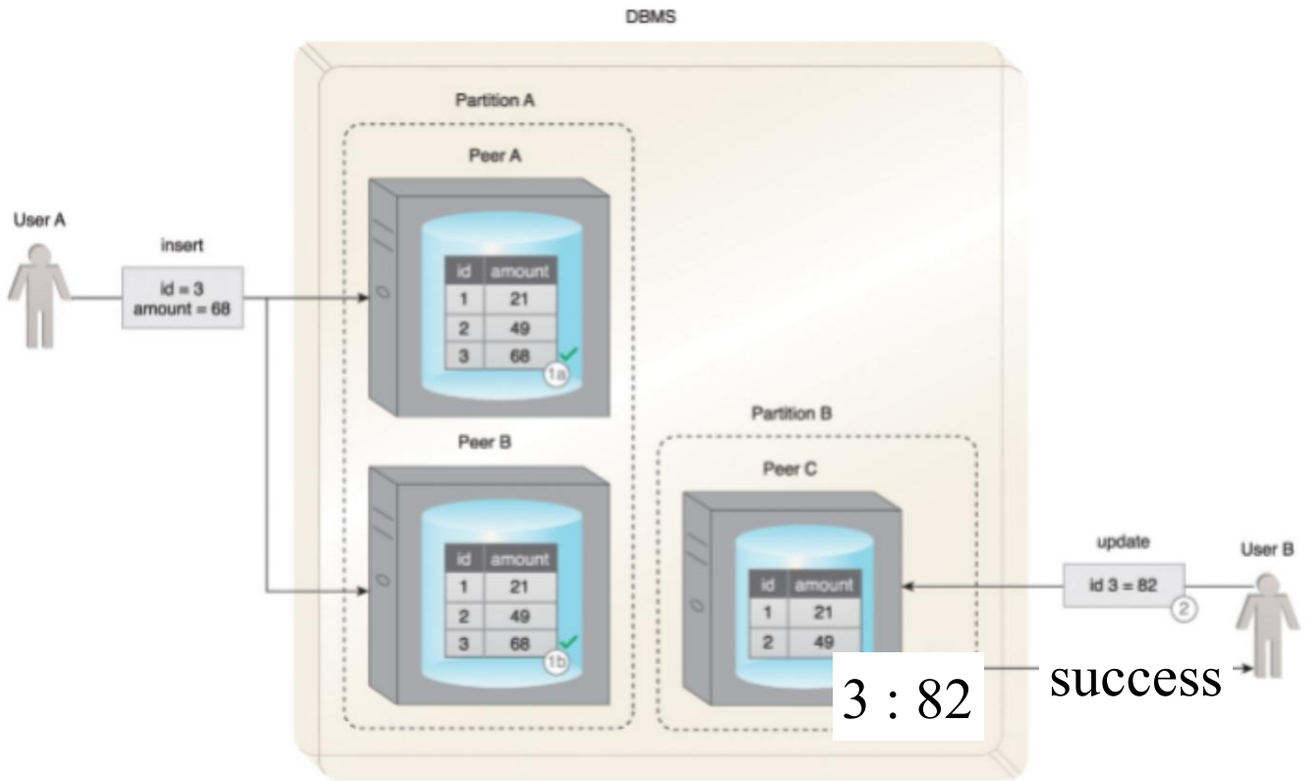


A network partition happens and user wants to update peer C

Option 1:

- Any access by user on peer C leads to failure message response
- System is available because it comes back with a response
- There is consistency because user's update is not processed
- But there is no partition tolerance
- C and A no P

P: Partition tolerance (2)

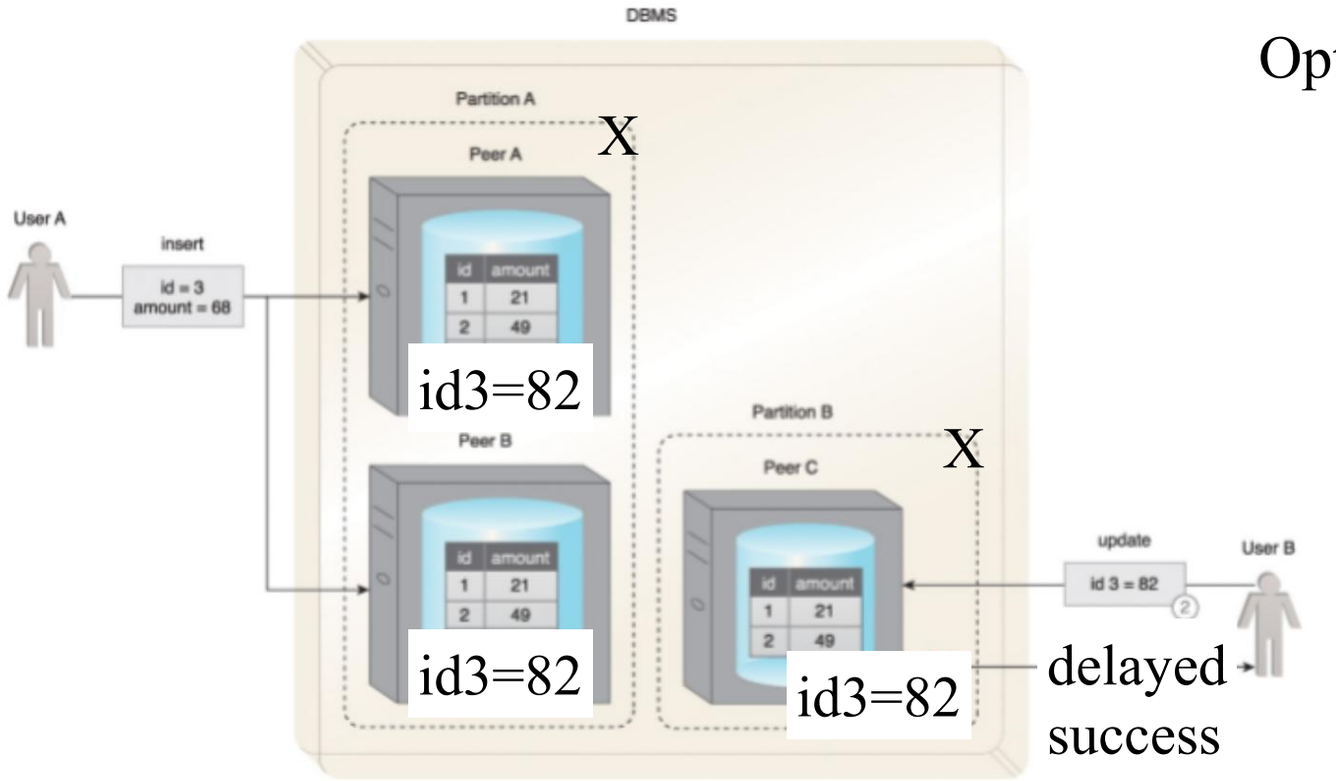


A network partition happens and user wants to update peer C

Option 2:

- Peer C records the update by user with success
- System is still available and now partition tolerant.
- But system becomes inconsistent
- P and A but no C

P: Partition tolerance (3)



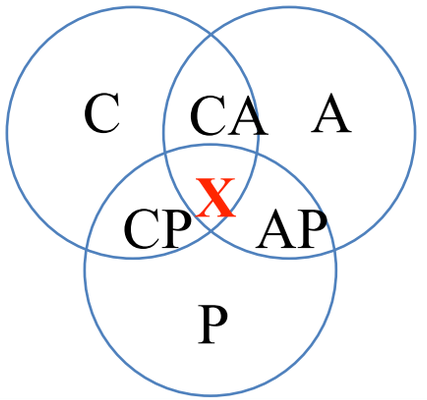
A network partition happens and user wants to update peer C

Option 3:

- User is made to wait till partition is fixed (could be quite long) and data replicated on all peers before a success message is sent
- System appears unavailable to user though it is partition tolerant and consistent
- P and C but no A

CAP Theorem (Brewer's Theorem)

- *A distributed data system, running over a cluster, can only provide two of the three properties C, A, P but not all.*
- So a system can be
  - CA or AP or CP but cannot support CAP
- In effect, when there is a partition, the system has to decide whether to pick consistency or availability.



Consistency or Availability choices

In a distributed database,

- Scalability and fault tolerance can be improved through additional nodes, although this puts challenges on maintaining consistency (C).
- The addition of nodes can also cause availability (A) to suffer due to the latency caused by increased communication between nodes.
  - May have to update all replicas before sending success to client . so longer takes time and system may not be available during this period to service reads on same data item.

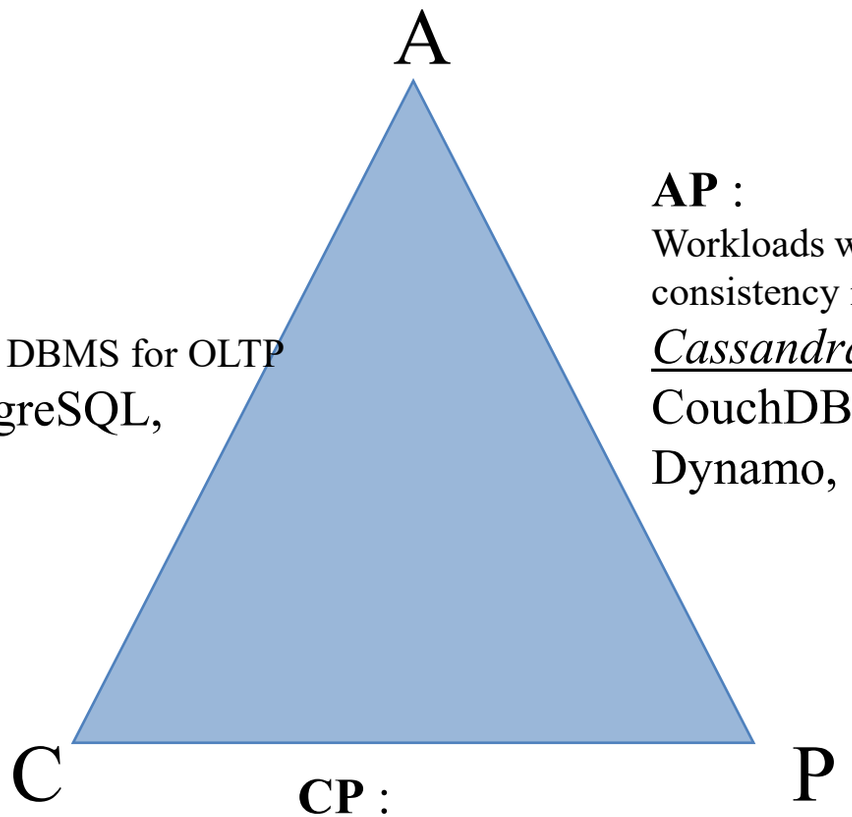
Large scale distributed systems cannot be 100% partition tolerant (P).

- Although communication outages are rare and temporary, partition tolerance (P) must always be supported by distributed database

Therefore, CAP is generally a choice between choosing either CP or AP

Traditional RDBMS systems mainly provide CA for single data items and then on top of that provide ACID for transactions that touch multiple data items.

Database Options



CA :
Traditional DBMS for OLTP
e.g. PostgreSQL, MySQL

AP :
Workloads where eventual consistency is sufficient
Cassandra, CouchDB, Dynamo,

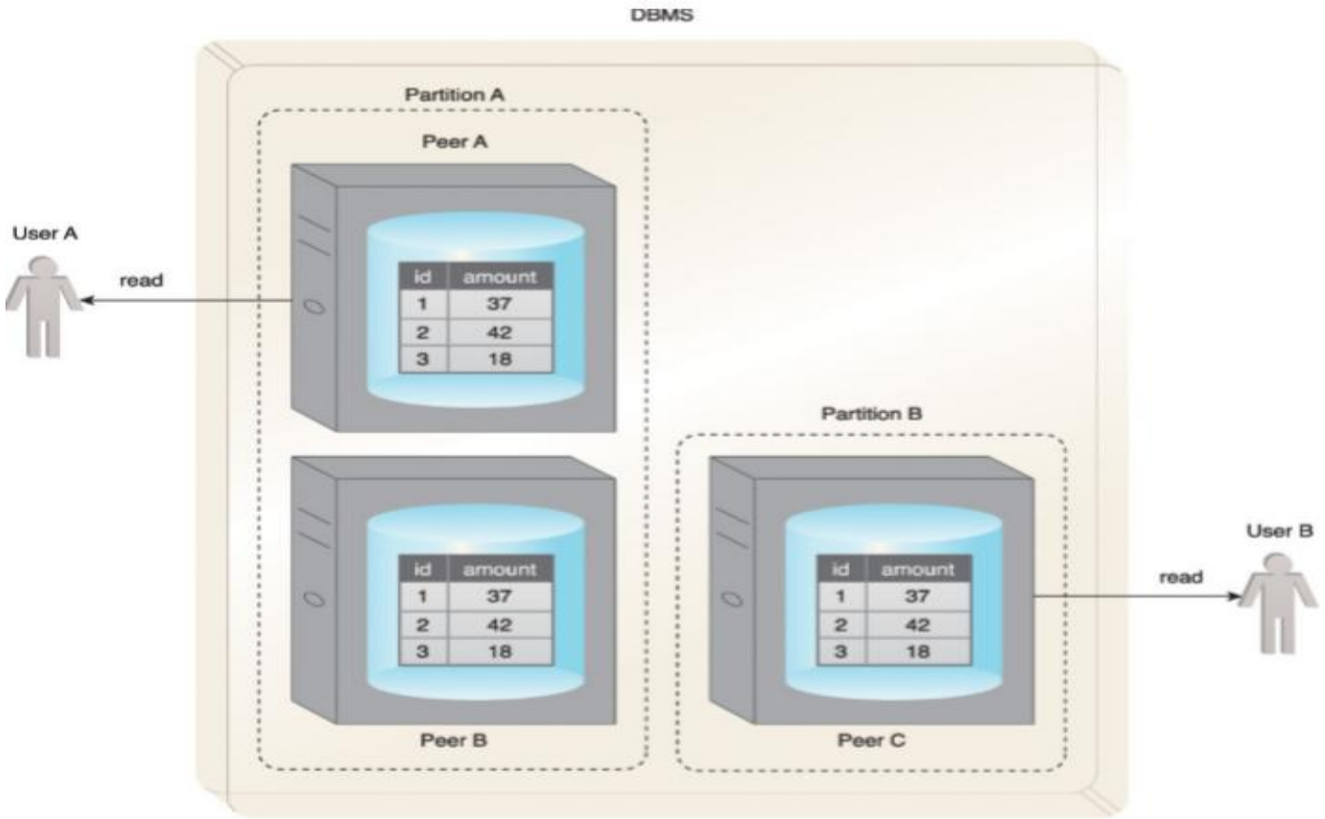
CP :
Can provide consistency with scale.
HBase, *MongoDB*, Redis

- Different design choices are made by Big Data DBs
- Faults are likely to happen in large scale systems
- Provides flexibility depending on use case to choose C, A, P behavior mainly around consistency semantics when there are faults

BASE

- BASE is a database design principle based on CAP theorem
- Leveraged by AP database systems that use distributed technology
- Stands for
  - ✓ Basically Available (BA)
  - ✓ Soft state (S)
  - ✓ Eventual consistency (E)
- It favours availability over consistency
- Soft approach towards consistency allows to serve multiple clients without any latency albeit serving inconsistent results
- Not useful for transactional systems where lack of consistency is concern
- Useful for write-heavy workloads where reads need not be consistent in real-time, e.g. social media applications, monitoring data for non-real-time analysis etc.

BASE – Basically Available



User A and User B receive data despite the database being partitioned by a network failure.

Database will always acknowledge a client's request, either in form of requested data or a failure notification

BASE – Soft State

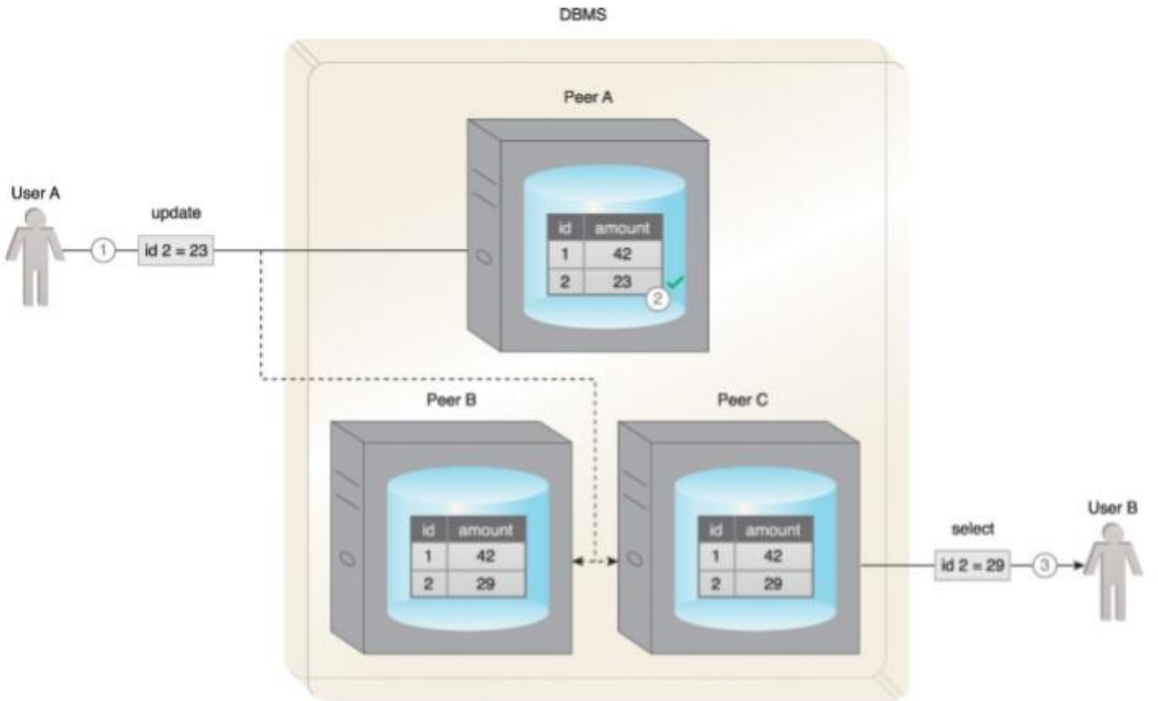
- Database may be in inconsistent state when data is read, thus results may change if the same data is requested again

  ✓ Because data could be uploaded for consistency, even though no user has written to the database between two reads

- User A updated record on node A
- Before the other nodes are updated,

  User B requests same record from C
- Database is now in a soft state and

  Stale data is returned to User B

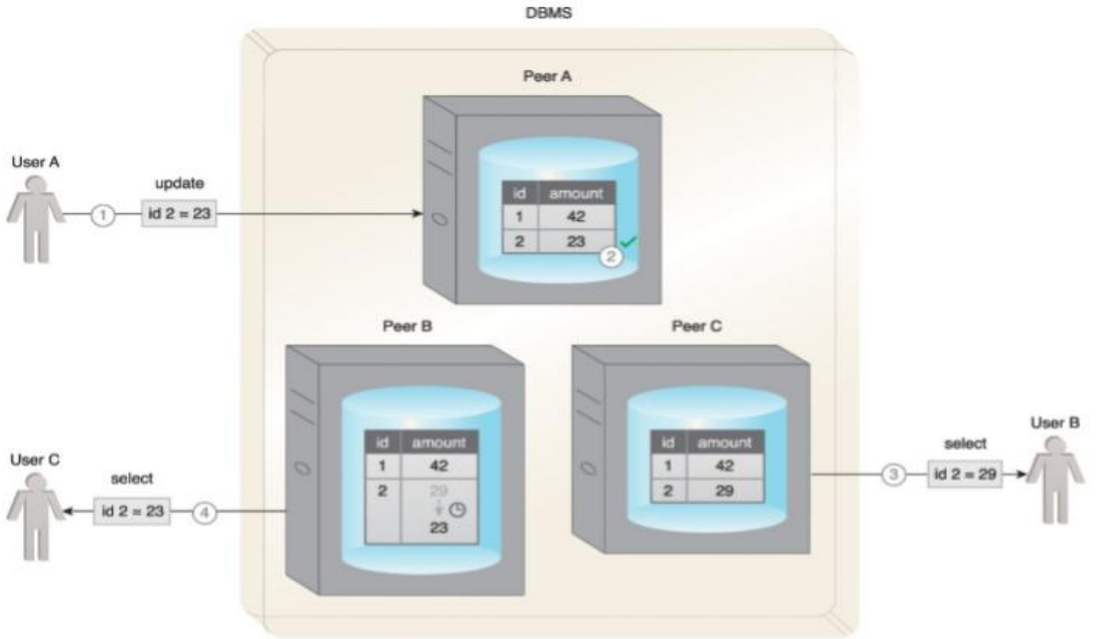


An example of the soft state property of BASE is shown here.

BASE – Eventual Consistency

- State in which reads by different clients, immediately following a write to database, may not return consistent results
- Database only attains consistency once the changes have been propagated to all nodes
- While database is in the process of attaining the state of eventual consistency, it will be in a soft state

✓ User A updates a record
✓ Record only gets updated at node A, but before other peers can be updated, User B requests data
✓ Database is now in soft state, stale data is returned to User B from peer C
✓ Consistency is eventually attained, User C gets correct v



An example of the eventual consistency property of BASE.

ACID vs BASE

- ACID and BASE represent two different approaches to managing **consistency, availability, and transactions** in database systems.

Aspect	ACID	BASE
Focus	Transaction correctness	Availability & scalability
Consistency	Strong consistency	Eventual consistency
Transaction Model	Strict transactional guarantees	More relaxed consistency guarantees
Data State	Consistent after transaction completion	May be temporarily inconsistent
Availability	May be reduced to preserve correctness	High availability prioritized
Scalability	Coordination can limit scalability	Designed for large distributed systems
Typical Use	Banking, payments, inventory	Social feeds, recommendations, large-scale web apps

Choosing Between ACID and BASE

The choice between ACID and BASE depends on the **business and application requirements**.

Choose ACID When...	Choose BASE When...
Data correctness is critical	High availability is critical
Immediate consistency is required	Temporary inconsistency is acceptable
Transactions involve critical updates	Workload is highly distributed
Incorrect data has serious consequences	Horizontal scalability is important

Summary – Distributed Transactions and Consistency Models

Key Takeaways

- **Distributed Transactions** involve operations across multiple nodes while maintaining a single logical transaction.
- **Two-Phase Commit (2PC)** coordinates participating nodes to achieve an **all-commit or all-abort** outcome.
- 2PC provides coordination but introduces **blocking, communication overhead, and performance challenges**.
- **Distributed Query Processing** executes queries across multiple nodes and combines their results.
- **Query Optimization** aims to reduce **data transfer, communication cost, and response time**.
- **CAP Theorem** highlights the trade-off between **Consistency and Availability during a network partition**.
- **BASE** favors availability and scalability while allowing **eventual consistency**.
- **ACID vs BASE** reflects different design priorities based on application requirements.

innovate achieve lead

Thank You